

Tecnatura Superior en Desarrollo de Software

Anexo 05d — Genéricos con `where` , Fluent Builder y CRTP

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué son estos tres conceptos y por qué aparecen juntos?

**where · Fluent Builder ·
CRTP**

Genéricos avanzados en
C#

Fundamento de GuardBuilderInterface

Al estudiar el código de los Guards encontramos una línea que puede resultar intimidante a primera vista:

```
1 public abstract class GuardBuilderInterface<TBu  
   ilder>  
2     where TBuilder : GuardBuilderInterface<TBu  
       ilder>
```

Esta única línea combina tres conceptos independientes que se necesitan mutuamente para lograr un diseño elegante y seguro. Entender cada uno por separado es el camino correcto antes de ver cómo se combinan.

Los tres conceptos:

- **where** — constraint de tipo genérico
- **Fluent Builder** — encadenamiento de métodos
- **CRTP** — tipo concreto en la clase base

1. Genéricos con `constraint where`

¿Qué tipos puede recibir un parámetro genérico? El compilador necesita garantías.

2. Fluent Builder

¿Cómo se encadenan métodos en una sola expresión? Cada método devuelve `this` — la misma instancia.

3. CRTP — Curiously Recurring Template Pattern

¿Por qué TBuilder aparece como argumento de su propia clase base? Permite que cada método devuelva el tipo concreto, no la clase base.

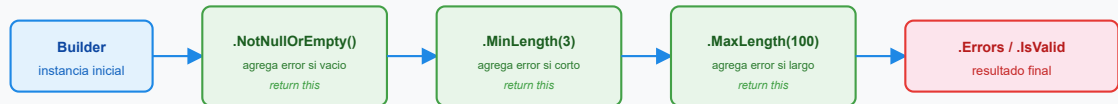
Analogía del mundo real

Imaginá una cadena de montaje en una fábrica de autos. Cada estación de trabajo recibe el auto, realiza una tarea específica y lo pasa a la siguiente estación. El auto que circula por la cadena es siempre **el mismo objeto** — no se crea uno nuevo en cada estación. Al final de la cadena el auto está completamente ensamblado.

El **Fluent Builder** funciona exactamente así: un objeto circula por una cadena de métodos, cada método le agrega algo y lo devuelve para que el siguiente método lo reciba. Al final de la cadena el objeto está completamente configurado.

Ahora bien, imaginá que la cadena puede fabricar autos o camiones. Cada vehículo tiene su propia cadena especializada, pero comparten estaciones comunes (pintura, control de calidad). El desafío es que cuando un auto pasa por una estación común y regresa, debe seguir siendo un auto — no un vehículo genérico. Eso es exactamente lo que resuelve el **CRTP**.

Cadena de montaje — el objeto circula y regresa siempre del mismo tipo



Cada metodo devuelve
el mismo objeto — no una copia, no la clase base

Características esenciales

1 — Genéricos con constraint `where`

Un parámetro de tipo genérico `T` puede representar cualquier tipo. Pero a veces el compilador necesita saber que `T` tiene ciertas características — que es una clase, que tiene constructor sin parámetros, o que hereda de un tipo específico. La palabra clave `where` expresa esa garantía.

```

1 // sin constraint - T puede ser cualquier cosa
2 // el compilador no sabe nada sobre T - no podemos llamar métodos específicos
3 public class Caja<T>
4 {
5     public void Mostrar(T item)
6     {
7         // item.Nombre → ERROR: T no garantiza tener esa propiedad
8         Console.WriteLine(item);
9     }
10 }
```

```

1 // con constraint where T : Producto
2 // el compilador sabe que T tiene todo lo que tiene Producto
3 public class Caja<T> where T : Producto
4 {
5     public void Mostrar(T item)
6     {
7         // ahora podemos acceder a cualquier miembro de Producto
8         Console.WriteLine(item.Nombre);
9     }
10 }
```

where T : Tipo le dice al compilador: "garantizo que T siempre será Tipo o alguna clase que herede de Tipo". A cambio, el compilador nos permite usar todos los miembros de Tipo sobre cualquier instancia de T — en tiempo de compilación, sin casteos.

Las constraints más comunes en C# son:

```
1  where T : class           // T debe ser un tipo referencia (clase)
2  where T : struct         // T debe ser un tipo valor (struct)
3  where T : new()           // T debe tener constructor sin parámetros
4  where T : ControllerBase // T debe heredar de ControllerBase
5  where T : IInterfaz      // T debe implementar IInterfaz
6  where T : notnull        // T no puede ser null
```

2 — El patrón Fluent Builder

El Fluent Builder es un patrón de diseño cuyo objetivo es permitir la configuración de un objeto mediante una cadena de llamadas a métodos — también llamada **API fluida**. Cada método realiza una acción sobre el objeto y devuelve `this` — la misma instancia — para que el siguiente método pueda ser invocado inmediatamente.

Sin Fluent Builder, configurar un objeto requiere múltiples líneas con referencias repetidas a la variable. Con Fluent Builder, toda la configuración se expresa en una sola expresión legible de izquierda a derecha.

SIN FLUENT BUILDER

Múltiples líneas — la variable se repite en cada una.

```
1  var consulta = new ConsultaLibros();
2  consulta.SetAutor("Borges");
3  consulta.SetAnioDesde(1940);
4  consulta.SetGenero("Fantastico");
5  consulta.SetMaxResultados(10);
```

CON FLUENT BUILDER

Una sola expresión encadenada — se lee como una oración.

```
1 var consulta = new ConsultaLibros()
2     .DeAutor("Borges")
3     .DesdeElAnio(1940)
4     .DelGenero("Fantastico")
5     .LimitarA(10);
```

La clave del Fluent Builder es que cada método devuelve `this` — la misma instancia sobre la que se llamó. Veamos la implementación mínima:

```
1 public class ConsultaLibros
2 {
3     private string _autor = "";
4     private int _limit = 20;
5
6     // devuelve this - permite encadenar el siguiente método
7     public ConsultaLibros DeAutor(string autor)
8     {
9         _autor = autor;
10        return this; // misma instancia - no se crea un objeto nuevo
11    }
12
13    // devuelve this - mismo patrón
14    public ConsultaLibros LimitarA(int max)
15    {
16        _limit = max;
17        return this;
18    }
19 }
```

3 — El problema que aparece al heredar un Fluent Builder

El Fluent Builder funciona perfectamente en una clase simple. El problema aparece cuando queremos heredar de ese Builder. Si la clase base devuelve `this` con el tipo de la clase base, el compilador ya no sabe que el objeto es del tipo concreto — y los métodos de la subclase desaparecen del encadenamiento.

```
1 // clase base que devuelve su propio tipo
2 public class BuilderBase
3 {
```

```
4      // devuelve BuilderBase – el tipo de la clase base
5      public BuilderBase ConOpcionComun()
6      {
7          return this;
8      }
9  }
10
11  // clase concreta que agrega métodos propios
12  public class BuilderString : BuilderBase
13  {
14      public BuilderString ConLongitudMinima(int min)
15      {
16          return this;
17      }
18  }
19
20  // el problema aparece al encadenar
21  new BuilderString()
22      .ConOpcionComun()      // devuelve BuilderBase – pierde el tipo
23      .ConLongitudMinima(3); // ERROR – BuilderBase no tiene este método
```

El problema: cuando un método de la clase base devuelve `BuilderBase`, el compilador "olvida" que el objeto es en realidad un `BuilderString`. Los métodos específicos de la subclase quedan inaccesibles — la cadena se rompe.

4 — CRTP: la solución elegante

El **CRTP** (Curiously Recurring Template Pattern) es un patrón en el que una clase pasa su propio tipo como argumento genérico a su clase base. Esto le permite a la clase base conocer el tipo concreto y devolver exactamente ese tipo — no el tipo de la base — en cada método.

```
1  // TBuilder es el tipo concreto que hereda de esta clase
2  // where garantiza que TBuilder siempre sea una subclase de BuilderBase
3  public abstract class BuilderBase<TBuilder>
4      where TBuilder : BuilderBase<TBuilder>
5  {
6      // devuelve TBuilder – el tipo concreto, no la clase base
7      public TBuilder ConOpcionComun()
8      {
9          // (TBuilder)this – el cast es seguro porque where lo garantiza
10         return (TBuilder)this;
11     }
```

```
12 }
13
14 // BuilderString pasa su propio tipo como argumento genérico
15 public class BuilderString : BuilderBase<BuilderString>
16 {
17     public BuilderString ConLongitudMinima(int min)
18     {
19         return this;
20     }
21 }
22
23 // ahora la cadena funciona correctamente
24 new BuilderString()
25     .ConOpcionComun() // devuelve BuilderString – tipo conservado
26     .ConLongitudMinima(3); // funciona – el tipo no se perdió
```

¿Por qué se usa?

Cada uno de los tres conceptos resuelve un problema concreto:

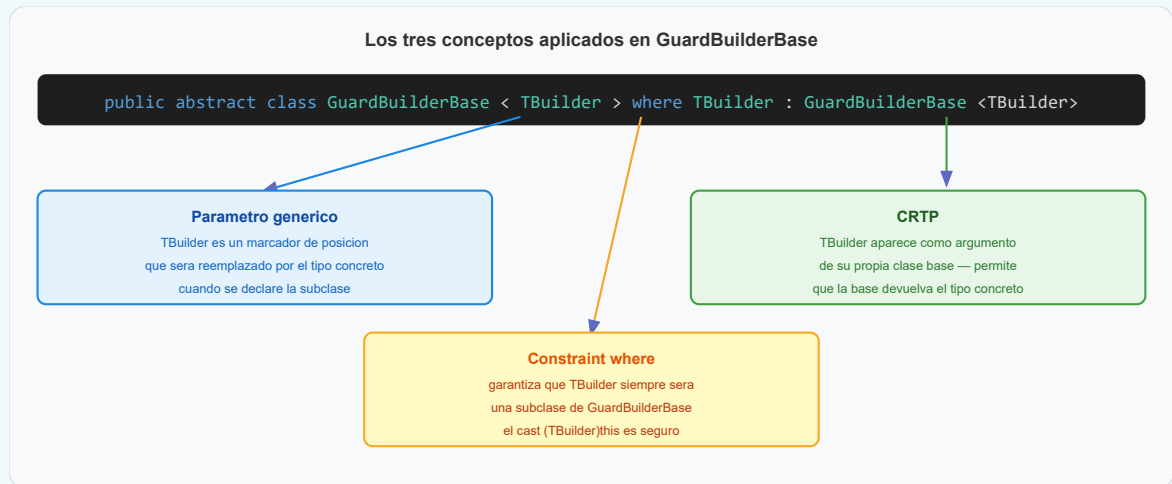
where — le da al compilador la información necesaria para verificar el código en tiempo de compilación, sin necesidad de casteos explícitos ni errores en tiempo de ejecución.

Fluent Builder — transforma configuraciones verbosas de múltiples líneas en expresiones únicas, legibles y expresivas que comunican la intención del código de forma directa.

CRTP — permite que una clase base sea reutilizable sin perder el tipo concreto en las cadenas de métodos. Sin él, heredar de un Fluent Builder rompería el encadenamiento cada vez que se llama a un método de la clase base.

¿Cómo encajan los tres conceptos en GuardBuilderInterface?

El diseño de `GuardBuilderInterface<TBuilder>` aplica los tres conceptos simultáneamente. El siguiente diagrama muestra cómo cada concepto contribuye a la línea que declaramos al principio:



La línea completa explicada:

```
public abstract class GuardBuilderInterface<TBuilder>
```

→ clase base genérica que acepta un parámetro de tipo TBuilder

```
where TBuilder : GuardBuilderInterface<TBuilder>
```

→ constraint CRTP: TBuilder debe ser una subclase de `GuardBuilderInterface<TBuilder>`

→ esto garantiza que `(TBuilder)this` es siempre seguro

→ y que los métodos de la base devuelven el tipo concreto, no la base

Marco teórico

Ejemplo completo ejecutable en Main()

El siguiente ejemplo implementa los tres conceptos juntos en su forma más simple y los prueba directamente en un programa de consola. No depende de ninguna otra clase del proyecto.

```
1 // paso 1 – clase base con CRTP y Fluent Builder
2 public abstract class ValidadorBase<TBuilder>
3     where TBuilder : ValidadorBase<TBuilder>
4 {
5     protected readonly List<string> _errores = new();
6     public IReadOnlyList<string> Errores => _errores;
7     public bool EsValido => _errores.Count == 0;
8
9     // método compartido – devuelve TBuilder gracias al CRTP
10    public TBuilder NoNulo(object? valor, string campo)
11    {
12        if (valor is null)
13            _errores.Add($"{campo} no puede ser nulo.");
14        return (TBuilder)this; // cast seguro – where lo garantiza
15    }
16 }
17
18 // paso 2 – subclase concreta que agrega reglas específicas
19 public class ValidadorTexto : ValidadorBase<ValidadorTexto>
20 {
21     private readonly string _valor;
22     public ValidadorTexto(string valor) => _valor = valor;
23
24     // método específico de texto – devuelve ValidadorTexto
25     public ValidadorTexto LongitudMinima(int min)
26     {
27         if (_valor?.Length < min)
28             _errores.Add($"Debe tener al menos {min} caracteres.");
29         return this;
30     }
31 }
32
33 // paso 3 – Main() prueba la cadena completa
34 var v = new ValidadorTexto("AB")
35     .NoNulo("AB", "Titulo") // método de la BASE – devuelve ValidadorTexto
36     .LongitudMinima(5);     // método de la SUBCLASE – funciona gracias al CRT
37 P
```

```
37
38 Console.WriteLine(v.EsValido); // False
39 foreach (var e in v.Errores)
40     Console.WriteLine(e);
41 // Salida: Debe tener al menos 5 caracteres.
```

Errores comunes

Error 1 — olvidar el constraint `where` en el CRTP

Sin el `where`, el compilador no acepta el cast `(TBuilder)this` porque no tiene garantía de que `TBuilder` sea compatible con el tipo actual.

ERROR COMUN

Sin `where` — el compilador rechaza el cast.

```
1 // falta el constraint – error de compilación
2 public abstract class BuilderBase<TBuilder>
3 {
4     public TBuilder ConOpcion()
5         => (TBuilder)this; // ERROR CS0030 – cast inválido
6 }
```

BUENA PRACTICA

Con `where` — el compilador acepta el cast.

```
1 // constraint presente – el cast es seguro
2 public abstract class BuilderBase<TBuilder>
3     where TBuilder : BuilderBase<TBuilder>
4 {
5     public TBuilder ConOpcion()
6         => (TBuilder)this; // correcto
7 }
```

Error 2 — no pasar el propio tipo en la herencia CRTP

Al declarar la subclase, el argumento genérico debe ser el tipo de la subclase misma — no la clase base ni otro tipo.

ERROR COMUN

Argumento genérico incorrecto — el tipo se pierde igual.

```
1 // MAL: pasa BuilderBase en vez de BuilderString
2 public class BuilderString
3     : BuilderBase<BuilderBase<BuilderString>>
4 { }
```

BUENA PRACTICA

El argumento genérico es el propio tipo concreto.

```
1 // BIEN: BuilderString pasa su propio tipo
2 public class BuilderString
3     : BuilderBase<BuilderString>
4 { }
```

Qué NO hacer

No usar el CRTP cuando la herencia no es necesaria

El CRTP agrega complejidad conceptual. Si la clase base no necesita devolver el tipo concreto — porque no habrá herencia o los métodos no se encadenan — el CRTP es sobrediseño innecesario.

QUE NO HACER

CRTP en una clase que nadie hereda — complejidad sin beneficio.

```
1 // innecesario si nadie hereda de ConsultaLibros
2 public class ConsultaLibros<T>
3     where T : ConsultaLibros<T> { }
```

BUENA PRACTICA

Sin herencia, Fluent Builder simple — devuelve el propio tipo directamente.

```
1 // simple y directo — sin CRTP
2 public class ConsultaLibros
3 {
4     public ConsultaLibros DeAutor(string autor)
5     {
6         return this;
7     }
8 }
```

Resumen — reglas de uso:

- Usar CRTP SOLO cuando exista herencia real con encadenamiento de métodos.
- El argumento genérico de la subclase SIEMPRE debe ser el propio tipo concreto.
- El constraint `where` NUNCA debe omitirse cuando se usa CRTP.
- Sin encadenamiento, usar `return this` directo — sin genéricos.